

The logo features the letters 'UML' in a bold, dark brown, sans-serif font. The text is centered within a white, cloud-like shape with a scalloped border. This white shape is set against a solid yellow background. A vertical dark brown bar is visible on the far left edge of the image.

UML

¿QUÉ ES UML?

- UML (Unified Modeling Language o Lenguaje Unificado de Modelado) es un lenguaje visual estandarizado para especificar, visualizar, construir y documentar sistemas..
- UML no es un lenguaje de programación ni una metodología de desarrollo
- Proporciona una notación compartida para comunicar decisiones de análisis y diseño.

UML: CARACTERÍSTICAS PRINCIPALES

- **Visual y gráfico.** Representa conceptos mediante símbolos y relaciones, reduciendo la ambigüedad del lenguaje natural.
- **Estandarizado.** La notación posee reglas comunes; esto facilita que distintos participantes interpreten el modelo.
- **Multiperspectiva.** Cada diagrama responde preguntas diferentes sobre la estructura, el comportamiento o la arquitectura.
- **Escalable.** Puede utilizarse en un ejercicio pequeño o en un sistema complejo, ajustando el nivel de detalle.
- **Independiente de la implementación.** Un modelo puede preceder a Java, C# u otro lenguaje, aunque ciertos elementos se traduzcan naturalmente a código.

UML: OBJETIVOS

- Comprender el problema antes de escribir código.
- Comunicar el diseño entre estudiantes, analistas, desarrolladores y usuarios.
- Reducir inconsistencias al hacer visibles clases, responsabilidades, dependencias e interacciones.
- Documentar decisiones relevantes para mantenimiento y evolución.
- Servir como puente entre requisitos, diseño y programación.

DIAGRAMAS ESTÁTICOS Y DINÁMICOS

Los modelos UML pueden agruparse didácticamente en dos grandes perspectivas. Los diagramas estructurales describen la organización relativamente estable del sistema; los diagramas de comportamiento muestran acciones, cambios e interacciones que ocurren a lo largo del tiempo.

Perspectiva	Pregunta central	Ejemplos UML	Analogía
Estática o estructural	¿Qué existe y cómo está organizado?	Clases, objetos, componentes, paquetes, despliegue	El plano de un edificio
Dinámica o de comportamiento	¿Qué sucede y en qué orden?	Secuencia, actividades, estados, casos de uso	La película de lo que ocurre

DIAGRAMAS DE CLASES

El diagrama de clases representa la estructura del dominio o del software: clases, atributos, operaciones, relaciones, multiplicidades y restricciones. Es uno de los diagramas centrales de UML y suele actuar como puente directo hacia la programación orientada a objetos.

LA CLASE Y SUS COMPARTIMIENTOS

Compartimiento	Contenido	Ejemplo
Nombre	Identidad de la clase; suele escribirse en singular y PascalCase.	Pedido
Atributos	Datos que describen el estado de cada objeto.	- estado: Estado
Operaciones	Servicios o comportamientos ofrecidos por la clase.	+ calcularTotal(): double

VISIBILIDAD Y FIRMA

Símbolo	Visibilidad	Uso habitual en Java
+	Public	Servicio accesible desde otras clases
-	Private	Atributo o detalle interno encapsulado
#	Protected	Acceso para la clase y sus subclases
~	Package	Acceso dentro del mismo paquete

Una operación puede escribirse como `nombre(parámetro: Tipo): TipoRetorno`.
Por ejemplo: `agregarProducto(producto: Producto, cantidad: int): boolean`.

RELACIONES ENTRE CLASE

Relación	Significado esencial	Notación / criterio
Asociación	Vínculo estable entre objetos que colaboran.	Línea continua; puede incluir nombre, roles, navegación y multiplicidades.
Agregación	Relación todo-parte débil; la parte puede existir separadamente.	Rombo blanco junto al todo.
Composición	Relación todo-parte fuerte; el todo controla el ciclo de vida de la parte.	Rombo negro junto al todo.
Generalización	Una subclase especializa a una superclase y hereda características.	Línea con triángulo vacío hacia la clase general.
Realización	Una clase concreta cumple el contrato de una interfaz.	Línea discontinua con triángulo vacío.
Dependencia	Uso puntual: un elemento necesita a otro para realizar una tarea.	Flecha discontinua hacia el elemento utilizado.

MULTIPLICIDADES

La multiplicidad expresa cuántas instancias pueden participar en cada extremo de una asociación. Se lee desde la clase opuesta: “para una instancia de A, ¿cuántas instancias de B pueden vincularse?”

Notación	Interpretación	Ejemplo
1	Exactamente una	Un pedido pertenece a un cliente
0..1	Cero o una	Un pedido puede tener un cupón
* o 0..*	Cero o muchas	Un cliente puede realizar pedidos
1..*	Una o muchas	Un pedido contiene al menos un ítem
m..n	Entre m y n	Un equipo posee entre 5 y 8 integrantes

DEL DIAGRAMA AL CÓDIGO JAVA

La traducción no siempre es mecánica, pero existe una correspondencia frecuente:

- una clase UML suele convertirse en una clase Java; los atributos privados en campos; las operaciones públicas en métodos; y una multiplicidad “muchos” en una colección o, en una primera aproximación didáctica, en un arreglo más un contador.

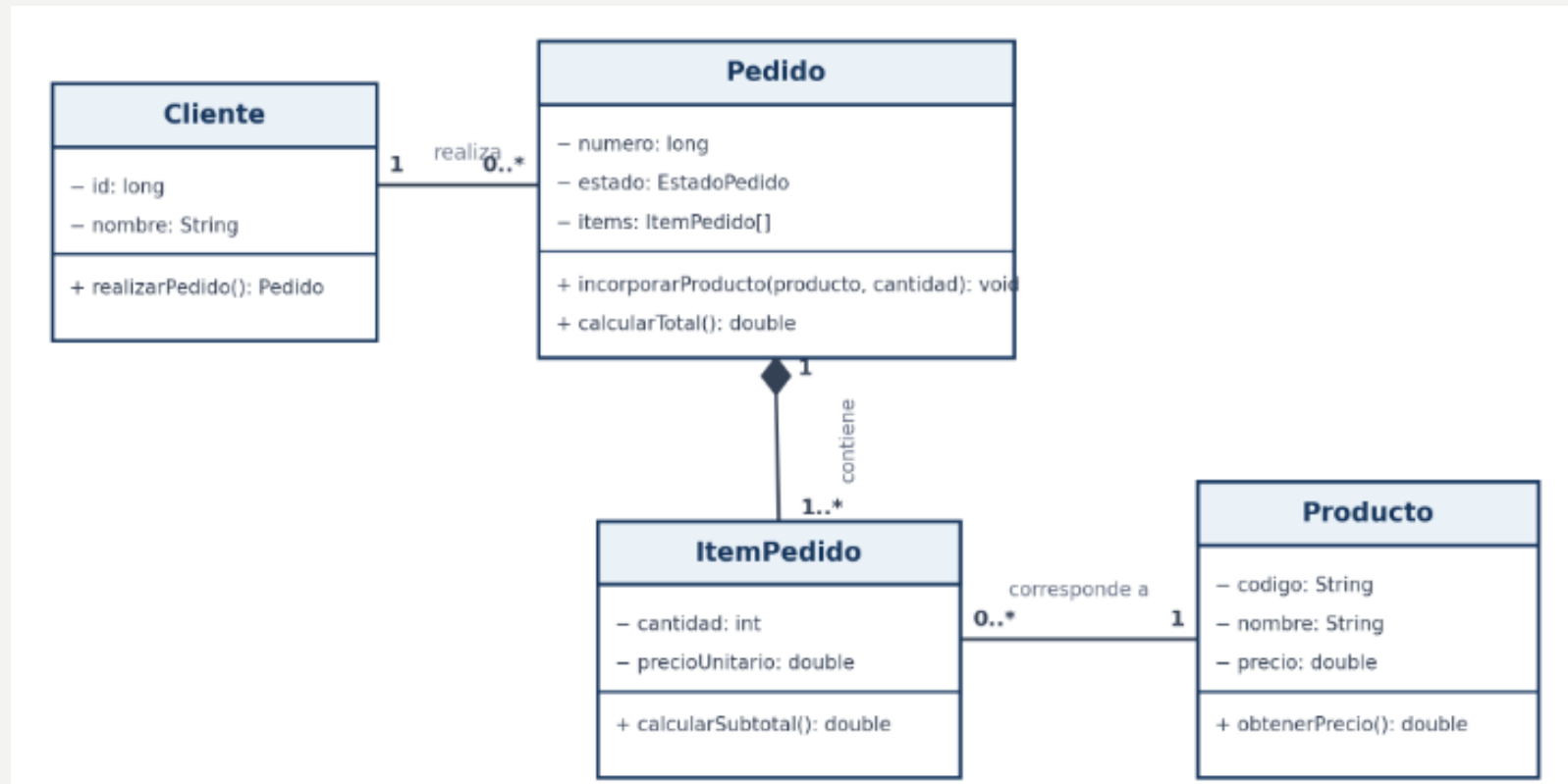
Criterio de diseño No conviene crear getters y setters indiscriminadamente. El modelo debe expresar responsabilidades significativas: `agregarProducto()`, `cambiarEstado()` o `calcularTotal()` comunican mejor la intención que exponer toda la estructura interna.

PROCEDIMIENTO PARA CONSTRUIRLO

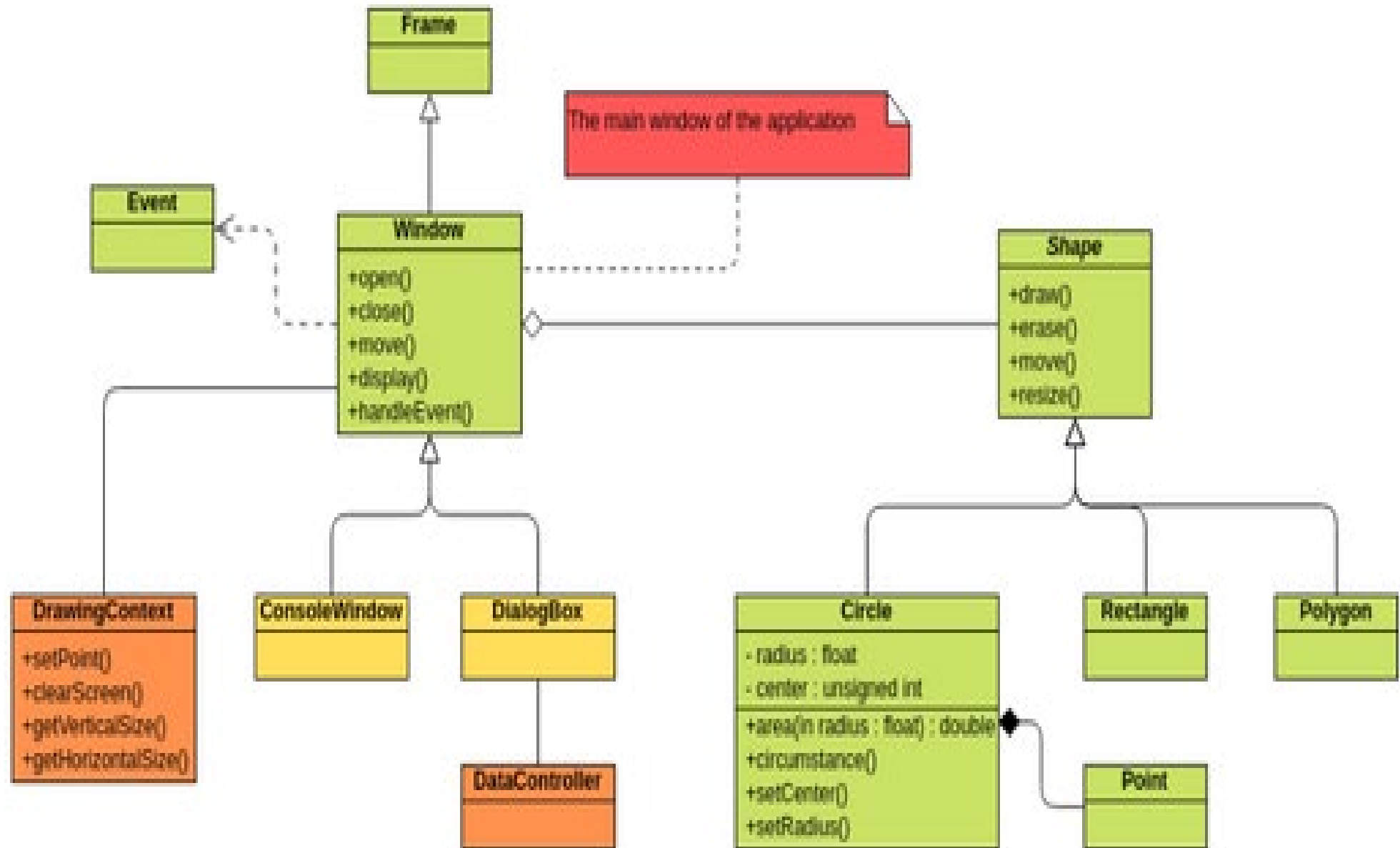
1. Leer la situación y subrayar conceptos relevantes del dominio.
2. Proponer clases candidatas; descartar términos que sean solo atributos, acciones o detalles irrelevantes.
3. Asignar responsabilidades: qué sabe y qué hace cada clase.
4. Identificar relaciones y justificar su semántica.
5. Determinar multiplicidades formulando preguntas en ambos sentidos.
6. Revisar nombres, visibilidad, navegabilidad y nivel de detalle.
7. Contrastar el modelo con escenarios de uso concretos.

EJEMPLO

Supongamos que un cliente puede realizar varios pedidos. Cada pedido posee un número, un estado y uno o más productos. El pedido ofrece operaciones para incorporar productos y calcular su total.



EJEMPLO



CASOS DE USO: CONCEPTO

El diagrama de caso de uso es un modelo que representa las funcionalidades principales de un sistema desde el punto de vista del usuario o actor externo.

- Su objetivo es mostrar qué hace el sistema, no cómo lo hace.
- Cada caso de uso describe una interacción entre un actor y el sistema que produce un resultado con valor para el usuario.

CASOS DE USO: IMPORTANCIA

Los diagramas de caso de uso son esenciales en las primeras etapas del análisis de requerimientos porque permiten:

- Comprender y comunicar qué funcionalidades espera el usuario del sistema.
- Servir de base para la especificación funcional.
- Facilitar la comunicación entre clientes, analistas y desarrolladores.
- Ayudar a definir el alcance del sistema (qué está dentro y fuera de sus límites).
- Ser el punto de partida para otros diagramas UML (como diagramas de secuencia o de clases).

CASOS DE USO: APLICACIÓN

Los diagramas de caso de uso se utilizan durante la **fase de análisis** en el desarrollo de software, especialmente cuando se trabaja con **metodologías orientadas a objetos** o **diseño centrado en el usuario**.

Aplicaciones típicas:

- En la **definición de requerimientos funcionales**.
- Para describir **procesos de negocio**.
- En la **documentación de sistemas existentes**.
- Como base para elaborar **pruebas funcionales**

CASOS DE USO: ELEMENTOS

Elemento	Descripción
Actor	Representa una persona, sistema o dispositivo que interactúa con el sistema. Se dibuja como un muñeco. Ejemplo: Cliente, Administrador, Sistema de Pago.
Caso de uso	Funcionalidad o servicio que el sistema ofrece a un actor. Se representa con una elipse. Ejemplo: Registrar usuario, Realizar pedido, Consultar saldo.
Sistema	Rectángulo que agrupa los casos de uso y define el límite del sistema.
Relación	Línea que conecta actores y casos de uso para indicar interacción.
Relaciones especiales	- «include»: un caso de uso incluye obligatoriamente a otro. - «extend»: un caso de uso extiende a otro bajo ciertas condiciones. - Generalización: indica herencia entre actores o entre casos de uso.

CASOS DE USO: EJEMPLO

- **Caso: “Sistema de Ventas Online”**

Actores:

- Cliente
- Administrador
- Sistema de Pago

Relaciones:

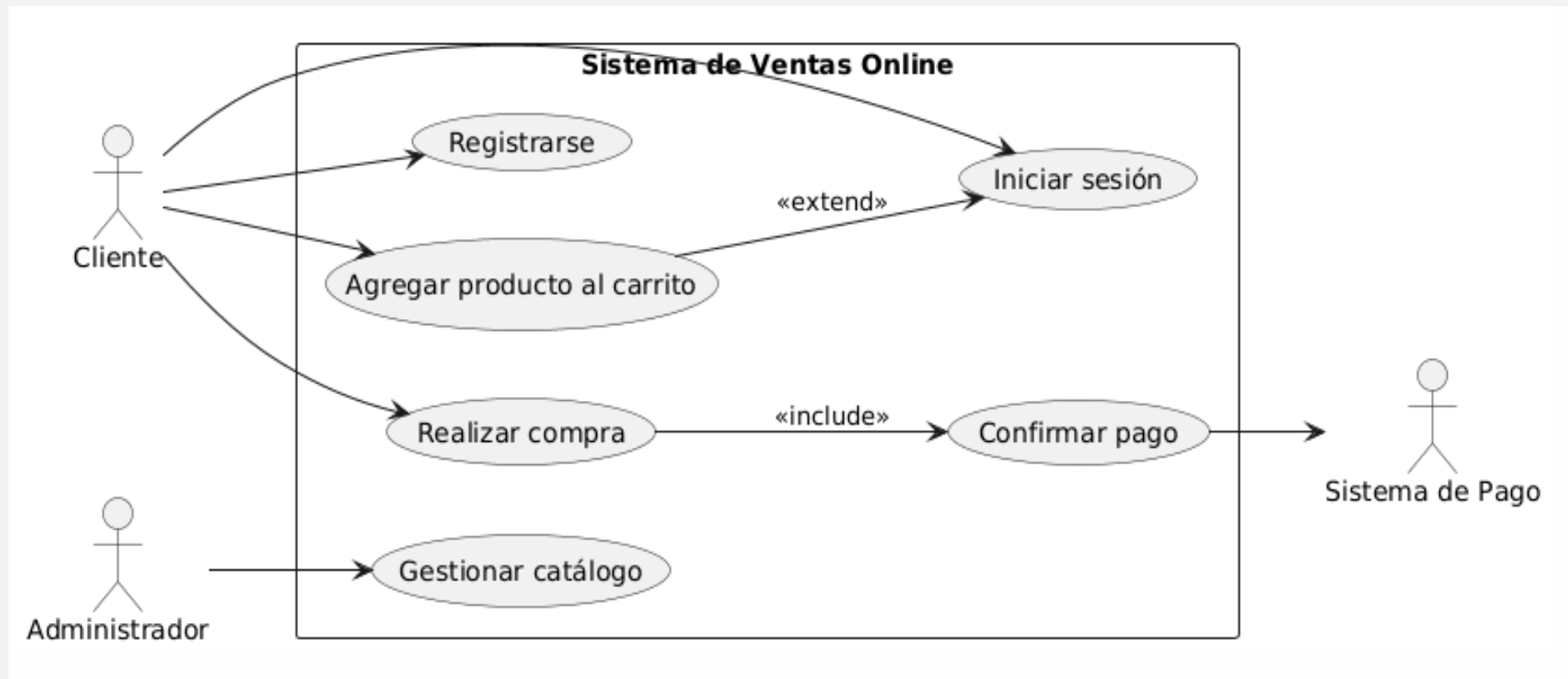
- Realizar compra **include** Confirmar pago
- Agregar producto al carrito **extend** Iniciar sesión (si el usuario no ha iniciado sesión, se le solicita hacerlo)

Casos de uso principales:

1. Registrarse en el sistema
2. Iniciar sesión
3. Agregar producto al carrito
4. Realizar compra
5. Confirmar pago
6. Gestionar catálogo (*solo para Administrador*)

CASOS DE USO: EJEMPLO

- Caso: “Sistema de Ventas Online”



CASOS DE USO: EJEMPLO 2

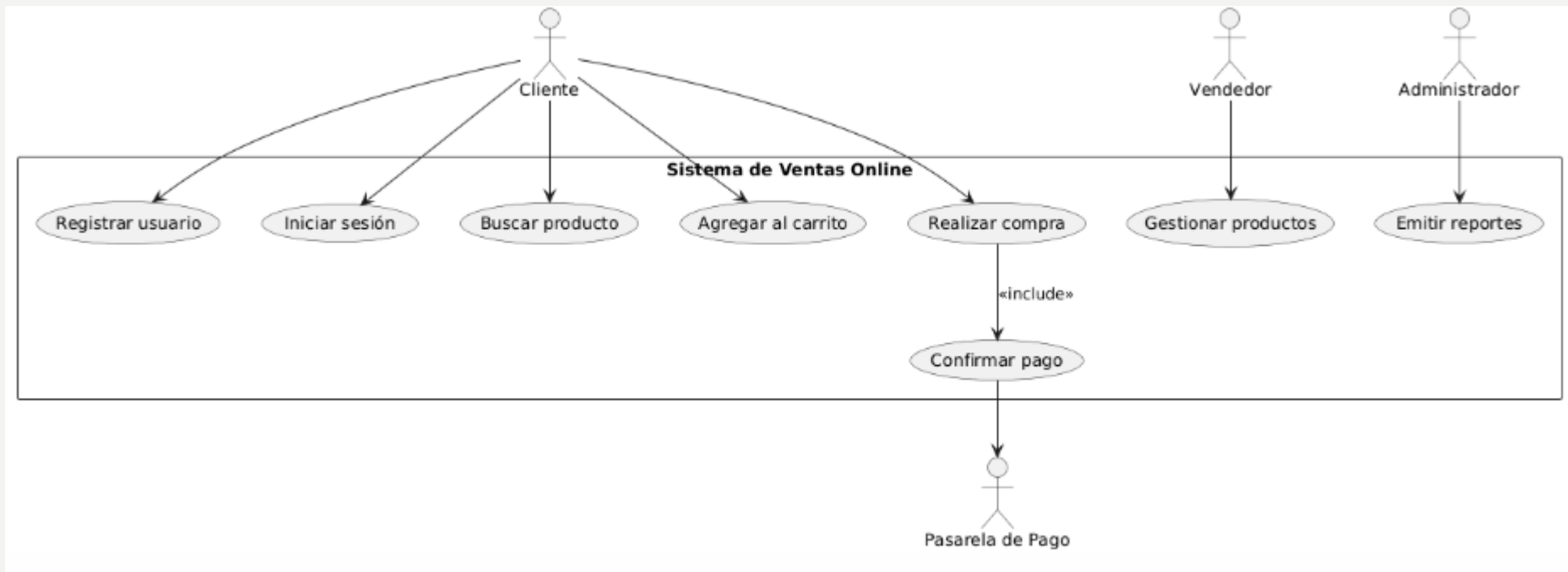


DIAGRAMA DE SECUENCIA: CONCEPTO

Es un tipo de diagrama de interacción del lenguaje UML (Unified Modeling Language) que muestra cómo los objetos se comunican entre sí mediante el intercambio de mensajes a lo largo del tiempo.

Su principal objetivo es visualizar la dinámica de un sistema, es decir, el orden en que ocurren las interacciones para cumplir un determinado comportamiento o caso de uso.

Mientras que un diagrama de clases muestra la estructura estática del sistema, el diagrama de secuencia representa el comportamiento dinámico: quién llama a quién, en qué orden y con qué mensajes.

DIAGRAMA DE SECUENCIA: IMPORTANCIA

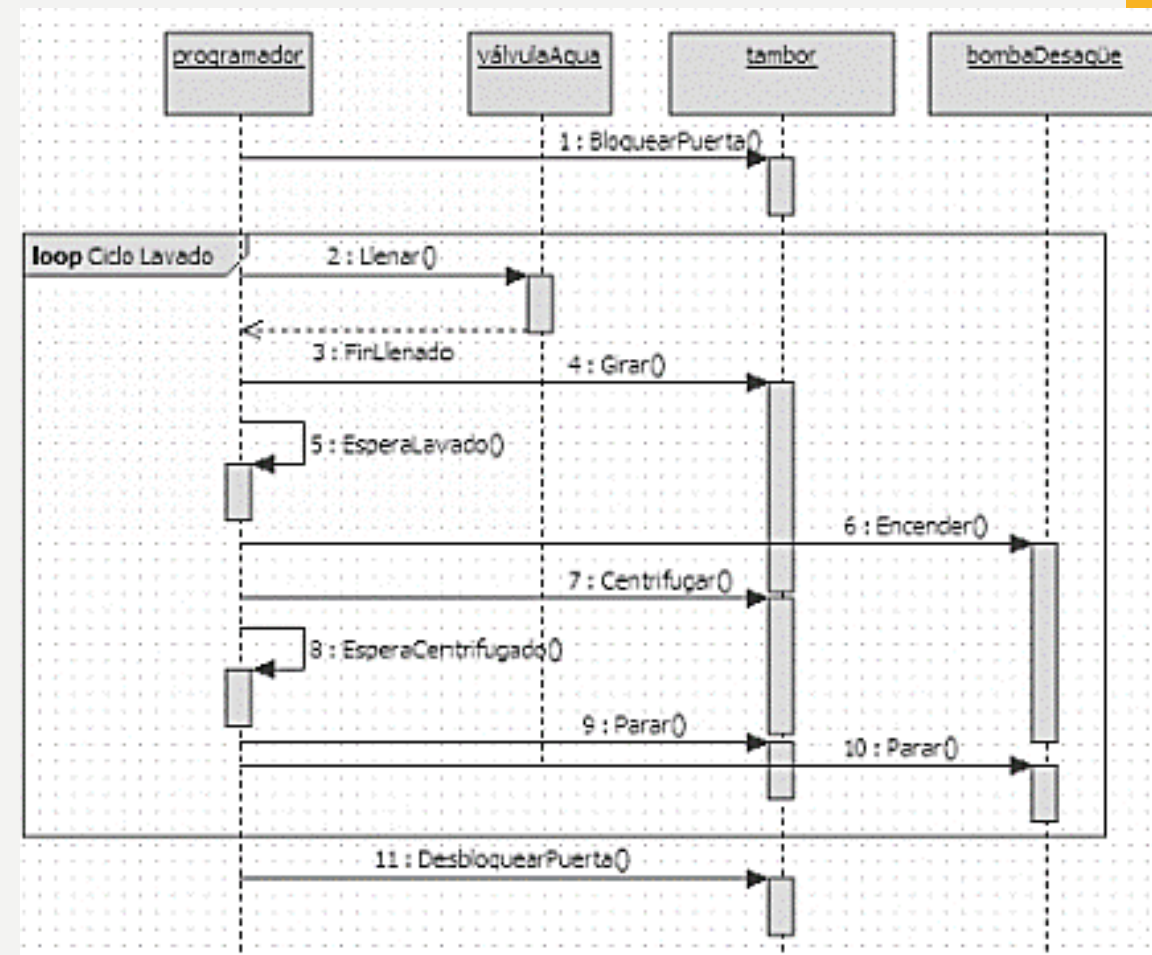
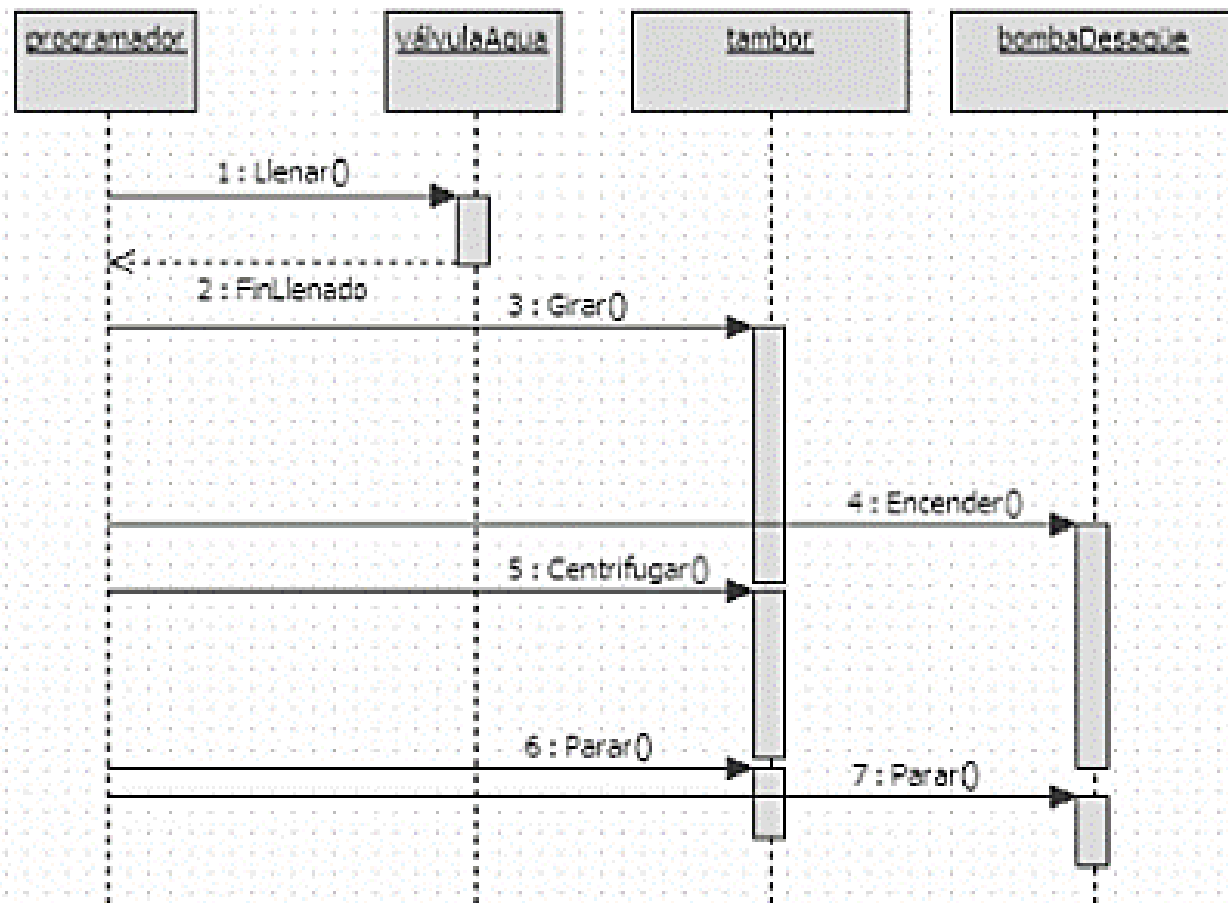
- Comprender y analizar la lógica de interacción entre los componentes del sistema.
- Documentar el flujo de eventos de un caso de uso o proceso.
- Detectar errores o ambigüedades antes de la implementación del código.
- Servir como puente entre el análisis y el diseño, facilitando la comunicación entre analistas, diseñadores y programadores.
- Favorecer la trazabilidad entre requisitos funcionales y componentes del sistema.

DIAGRAMA DE SECUENCIA: APLICACIÓN

- Se utilizan principalmente en las etapas de análisis y diseño de sistemas orientados a objetos, por ejemplo:
- Para representar el flujo de mensajes en un caso de uso.
- Para definir la lógica de métodos complejos.
- Para modelar interacciones entre subsistemas o servicios, en arquitecturas distribuidas (como microservicios o sistemas cliente-servidor).
- En la validación del comportamiento esperado antes de escribir código.

ELEMENTOS QUE LO COMPONEN

Elemento	Descripción
Actor	Representa un usuario u otro sistema externo que interactúa con el sistema.
Objeto / Instancia	Entidad que participa en la interacción. Se representa como un rectángulo con su nombre subrayado. Ejemplo: :Cliente, :ControladorVenta.
Línea de vida (lifeline)	Línea vertical que indica el paso del tiempo y la existencia del objeto durante la interacción.
Mensaje	Flecha que conecta dos líneas de vida, representando una llamada a un método, el envío de un mensaje o una respuesta.
Activación	Rectángulo angosto sobre la línea de vida que indica cuándo un objeto está ejecutando una acción.
Retorno de mensaje	Flecha discontinua que indica la devolución de un valor o la finalización de un mensaje.
Bloques de control	Elementos opcionales que permiten mostrar condiciones (alt), bucles (loop), o paralelismos (par).



EJEMPLO:

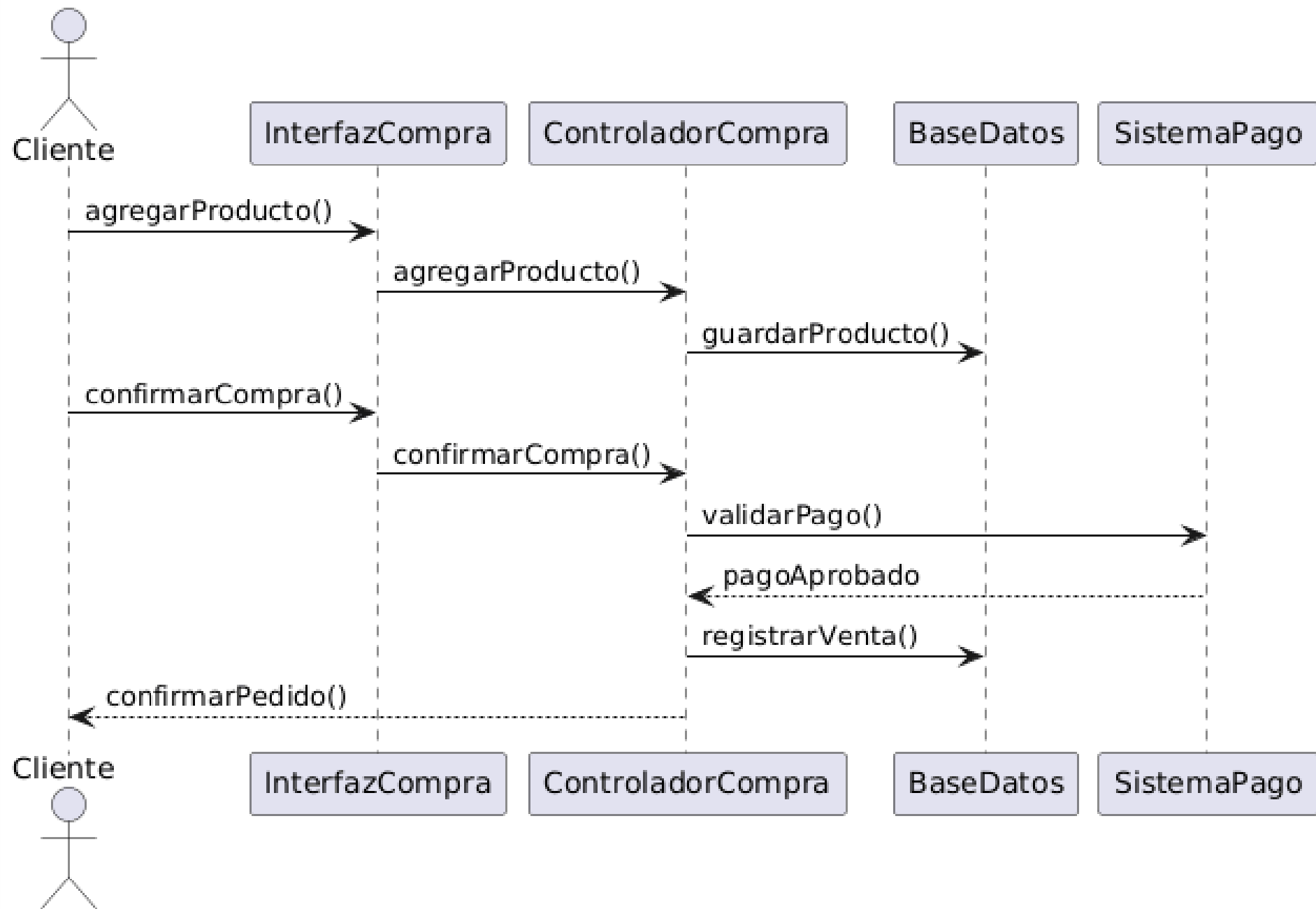
Un cliente selecciona un producto, lo agrega al carrito y confirma la compra. El sistema valida el pago y confirma el pedido.

Participantes:

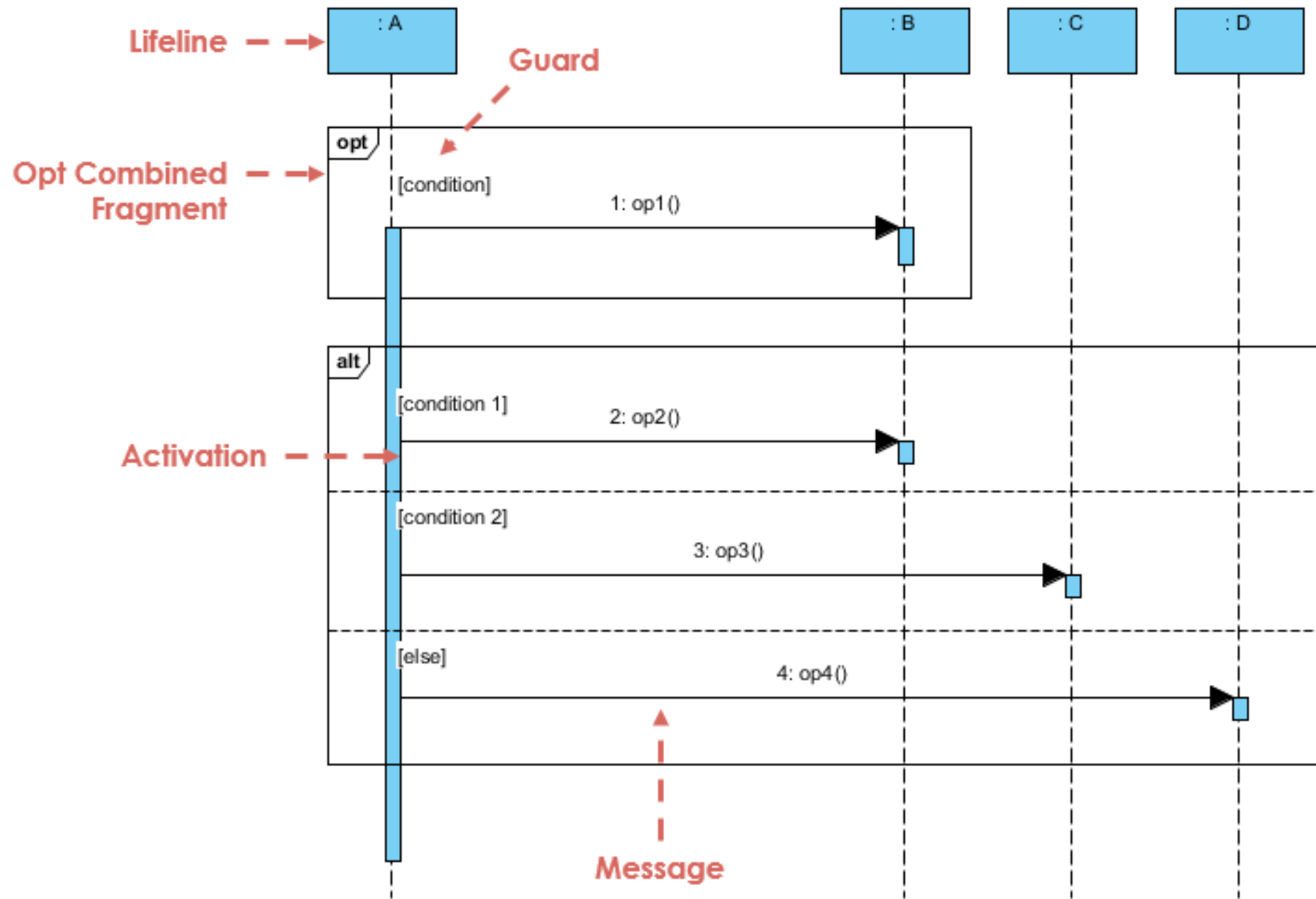
- Cliente
- InterfazCompra
- ControladorCompra
- SistemaPago
- BaseDatos

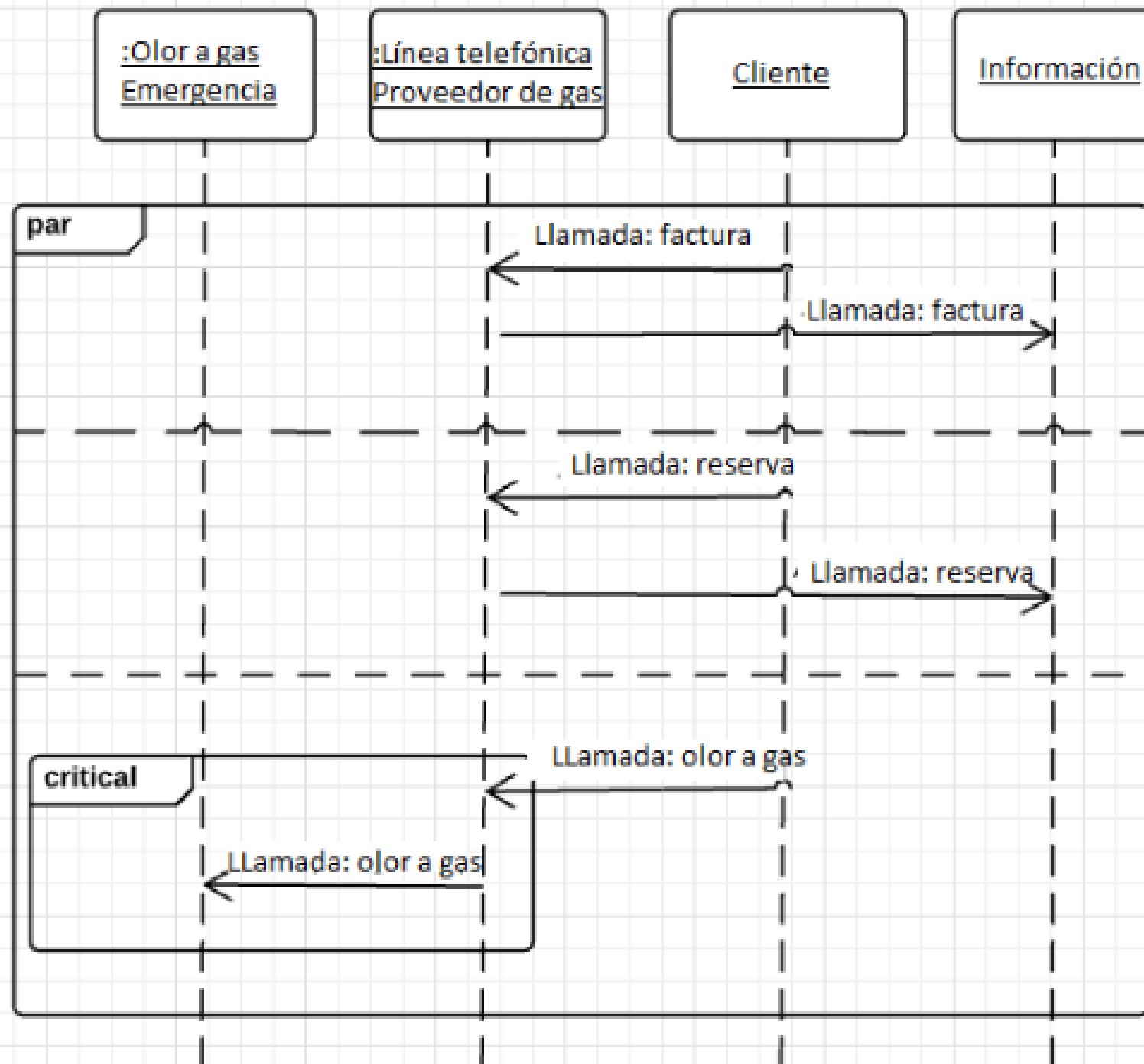
Flujo de interacción:

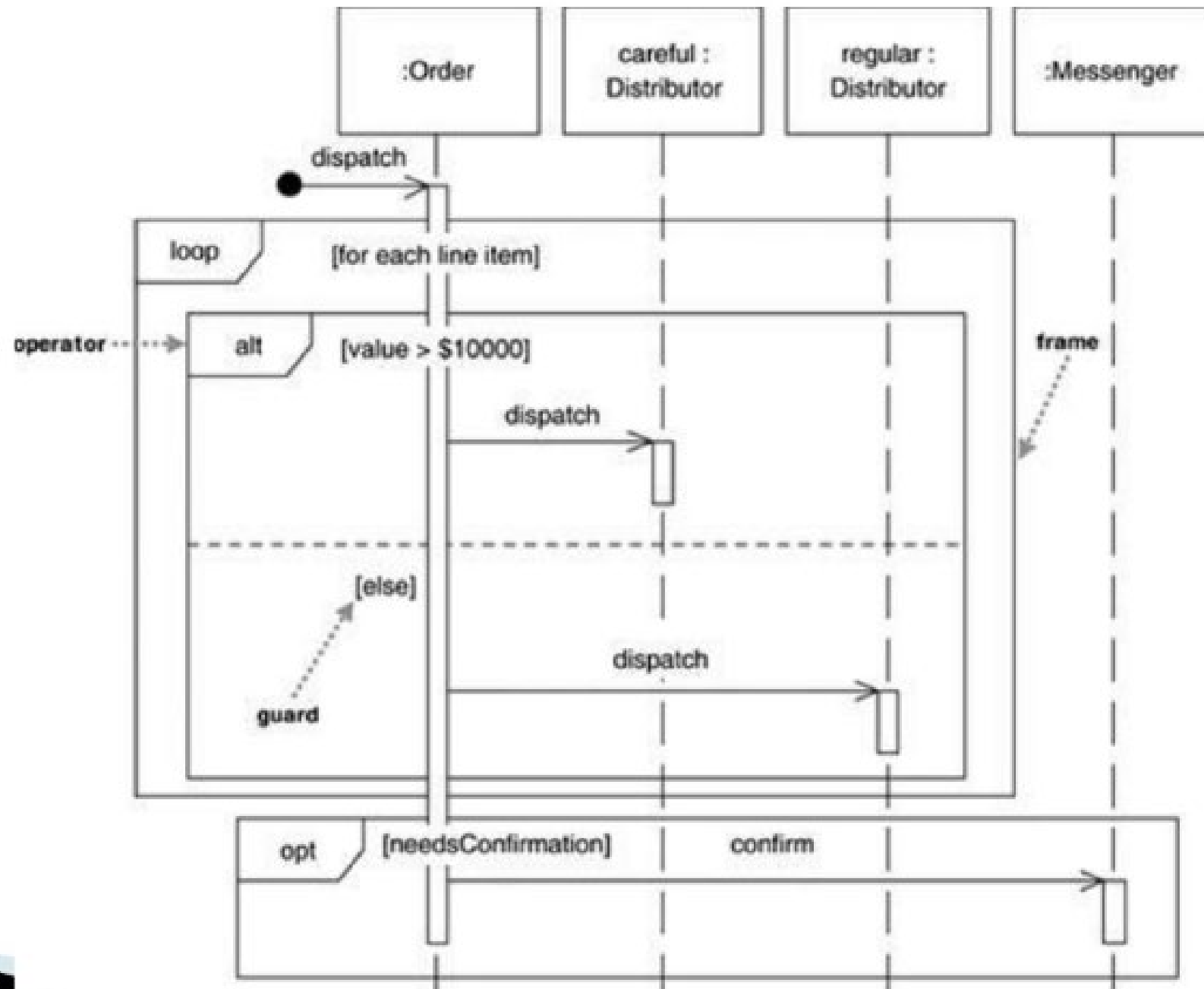
- El Cliente solicita agregar un producto → agregarProducto().
- La InterfazCompra envía el mensaje al ControladorCompra.
- El ControladorCompra guarda el producto en la BaseDatos
- .El Cliente confirma la compra → confirmarCompra().
- El ControladorCompra solicita autorización al SistemaPago.
- Si el pago es válido, el ControladorCompra registra la venta y envía confirmación al Cliente.



OTROS EJEMPLOS







TAREA:

Se desea modelar el módulo de préstamos de una biblioteca. Un socio puede buscar libros en el catálogo, solicitar el préstamo de un libro disponible, renovarlo y devolverlo. Un bibliotecario se encarga de registrar nuevos socios y de gestionar el catálogo de libros. Cuando un socio solicita un préstamo, el sistema verifica la disponibilidad del libro en la base de datos y, si está disponible, registra el préstamo.